# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 2 – Case Study

| | | | |
|---|---|---|---|
| **Course Code:** | DSA03 | **Course Title:** | Natural Language Processing |
| **CO Assessed:** | CO3 – Precision | **Assessment:** | Assessment Tool 2 – Case Study |
| **Weightage:** | 30% of CIA | | |
| **CO5 Statement:** | Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3) | | |
| **Student Name:** | M.Rishitha | **Reg. No.:** | 192424335 |
| **Section / Batch:** | 2024 AIDS | **Date:** | 16-08-2026 |

**Code of Conduct**

*I ____M.Rishitha[192424335]_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: __M.Rishitha_____

**Instructions**

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

| Section / Topic | Focus | Case Study |
|---|---|---|
| N-Gram Language Models and Smoothing Techniques | Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction | Case Study 1 – Smart Mobile Keyboard Prediction System |
| Part-of-Speech (POS) Tagging and Word Classes | Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding | Case Study 2 – AI-Powered Customer Support Chatbot |
| Transformation-Based Tagging (Brill Tagger) | POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling | Case Study 3 – News Analytics and POS Tag Correction System |
| Corpus Statistics and Probabilistic Analysis | Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation | Case Study 3 – News Analytics and POS Tag Correction System |
| Integrated Syntactic Analysis and NLP Applications | Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems | Case Studies 1, 2, and 3 |

## CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

*"machine learning improves business, machine learning enables automation, machine learning drives innovation."*

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- C(machine) = 3
- C(machine learning) = 3
- C(learning improves) = 1
- C(learning enables) = 1
- C(learning drives) = 1
- C(learning improves business) = 1

The interpolation weights are:

- $\lambda_1$ = 0.5 (Trigram)
- $\lambda_2$ = 0.3 (Bigram)
- $\lambda_3$ = 0.2 (Unigram)

The prediction probabilities after the word **"learning"** are:

- P(improves) = 0.33
- P(enables) = 0.33
- P(drives) = 0.33

### Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **P(learning | machine)**. Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **P(improves) = 0.33, P(enables) = 0.33, P(drives) = 0.33**. Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

**CASE STUDY 2: AI-Powered Hospital Appointment Chatbot**

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- P(book | VB) = 0.7
- P(book | NN) = 0.3
- P(Start → VB) = 0.6
- P(Start → NN) = 0.4

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

**Questions**

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

**CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis**

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

**"Market growth drives investment."**

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

**Change NNS to VBZ if the preceding word is tagged as NN.**

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

**Questions**

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

## Rubrics for Calculation

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Case | 25 | Thorough understanding; clearly identifies all key issues | Good understanding with most issues identified | Basic understanding; some issues missed | Poor understanding; problem not identified correctly |
| Application of Concepts | 25 | Effectively applies relevant concepts to analyze the case deeply | Applies appropriate concepts with minor gaps | Limited application of concepts | Fails to apply relevant concepts |
| Analysis | 20 | In-depth analysis with strong reasoning and insights | Good analysis with logical reasoning | Basic analysis with limited depth | Weak or no analysis; lacks reasoning |
| Solution Design | 20 | Innovative, feasible solutions with strong justification | Logical solutions with adequate justification | Basic solutions with weak justification | Incorrect or no solution provided |
| Clarity | 10 | Clear, well-structured, and easy to understand | Organized and understandable presentation | Limited clarity and structure | Poor clarity; difficult to understand |

| Q. No. / Criteria Level | Understanding of Case | Application of Concepts | Analysis | Solution Design | Clarity | Sub. Total | Total % |
|---|---|---|---|---|---|---|---|
| 1 | 7 | 7 | 7 | 6 | 6 | 33 | 95 |
| 2 | 7 | 7 | 7 | 7 | 6 | 34 | |
| 3 | 6 | 6 | 6 | 5 | 5 | 28 | |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

ASSESSMENT TOOL 2 – CASE STUDY

RegisterNo:192424335

Name: M.Rishitha Date:17-08-2026

COURSE: Natural language processing Code: DSA0306

---

## CASE STUDY 1 – E-COMMERCE SMART SEARCH AND PRODUCT PREDICTION SYSTEM

### Introduction

An e-commerce search and autocomplete system uses N-gram language models to predict the next word in a customer's query. The given corpus is:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The system uses Bigram and Trigram Language Models, Maximum Likelihood Estimation (MLE), Backoff Models, Deleted Interpolation, and Entropy Analysis.

Given:

- C(machine) = 3
- C(machine learning) = 3
- C(learning improves) = 1
- C(learning enables) = 1
- C(learning drives) = 1
- C(machine learning improves) = 1

Interpolation weights:

- $\lambda_1 = 0.5$ for Trigram
- $\lambda_2 = 0.3$ for Bigram
- $\lambda_3 = 0.2$ for Unigram

Prediction probabilities after "learning":

- P(improves) = 0.33
- P(enables) = 0.33
- P(drives) = 0.33

### Q1. Compute the MLE Bigram Probability P(learning | machine)

For a bigram language model:

$$
P(w_i|w_{i-1}) = \frac{C(w_{i-1},w_i)}{C(w_{i-1})}
$$

Therefore:

$$
P(\text{learning}|\text{machine}) \frac{C(\text{machine},\text{learning})}{C(\text{machine})}
$$

Substituting the given values:

$$
P(\text{learning}|\text{machine}) \frac{3}{3}
$$

$$
\boxed{P(\text{learning}|\text{machine})=1.0}
$$

Interpretation

The probability is 1 because every occurrence of "machine" in the given corpus is followed by "learning".

Therefore, when a customer types:

machine

the language model gives a very high confidence that the next word should be:

learning

This makes the autocomplete system more accurate for this particular phrase

**Q2. Backoff Model for the Unseen Sequence "machine learning transforms"**

The sequence:

machine learning transforms

contains the unseen word sequence "learning transforms".

The trigram:

machine learning transforms

does not occur in the corpus.

The bigram:

learning transforms

also does not occur.

Therefore, the model backs off to a lower-order N-gram.

Backoff process

machine learning transforms

↓

Trigram unavailable

↓

learning transforms

↓

Bigram unavailable

↓

P(transforms)

↓

Unigram model

The word transforms does not occur in the supplied corpus.

Therefore:

$$
C(transforms)=0
$$

and the unsmoothed unigram MLE is:

$$
P(transforms)=
\frac{C(transforms)}
{C(total)}

\frac{0}{12}

0
$$

Thus:

$$
\boxed{P(transforms)=0}
$$

Interpretation

The supplied corpus contains no occurrence of "transforms", so even after backing off to the unigram level, the unsmoothed probability is zero.

In a practical search system, this problem would normally be handled using a smoothing method such as Laplace smoothing, Good-Turing smoothing, or an unknown-word probability.

Importance of Backoff

Backoff is useful because users can enter queries that were not observed during training. Instead of immediately failing at the trigram level, the system attempts to use lower-order information.

Therefore, backoff improves the robustness of an N-gram-based autocomplete system.

**Q3. Deleted Interpolation Probability of "machine learning improves"**

Deleted interpolation combines trigram, bigram, and unigram probabilities:

[
P_{interp}

\lambda_1P_{trigram}
+
\lambda_2P_{bigram}
+
\lambda_3P_{unigram}
]

Given:

[
\lambda_1=0.5
]

[
\lambda_2=0.3
]

[
\lambda_3=0.2
]

Step 1 – Trigram probability

For:

machine learning improves

[
P(improves|machine,learning)

\frac{C(machine,learning,improves)}
{C(machine,learning)}
]

Given:

[
C(machine,learning,improves)=1
]

[
C(machine,learning)=3
]

Therefore:

[
P_{trigram}=\frac{1}{3}=0.3333
]

Step 2 – Bigram probability

For:

learning improves

[
P(improves|learning)

\frac{C(learning,improves)}
{C(learning)}
]

The word "learning" occurs 3 times and "learning improves" occurs once.

Therefore:

[
P_{bigram}=\frac{1}{3}=0.3333
]

Step 3 – Unigram probability

The corpus contains 12 words in total:

- machine learning improves business
- machine learning enables automation
- machine learning drives innovation

The word "improves" occurs once.

Therefore:

$$
P(improves)=\frac{1}{12}=0.0833
$$

Step 4 – Deleted interpolation

$$
P = (0.5)(0.3333) + (0.3)(0.3333) + (0.2)(0.0833)
$$

$$
P = 0.16665+0.09999+0.01666
$$

$$
\boxed{P\approx0.2833}
$$

Interpretation

Deleted interpolation combines information from three levels:

- Trigram → most contextual
- Bigram → less contextual
- Unigram → general word frequency

This makes the model more robust when some N-gram counts are sparse or unavailable.

**Q4. Entropy of the Prediction Distribution**

Given:

$$
P(improves)=0.33
$$

$$
P(enables)=0.33
$$

$$
P(drives)=0.33
$$

Entropy is calculated as:

$$
H(X)=-\sum p(x)\log_2p(x)
$$

Therefore:

$$
H=
-[0.33\log_2(0.33)
+0.33\log_2(0.33)
+0.33\log_2(0.33)]
$$

$$
H=-3(0.33\log_2(0.33))
$$

Since:

$$
\log_2(0.33)\approx-1.599
$$

$$
H\approx3(0.5277)
$$

$$
\boxed{H\approx1.58\ bits}
$$

The theoretical maximum for three equally likely outcomes is:

$$
\log_2(3)\approx1.585
$$

Interpretation

The entropy is close to the maximum possible value because the three possible next words have almost equal probabilities.

Therefore, the system has high uncertainty about which word should be predicted next.

---

Q5. Python Program

```
import nltk

import numpy as np

import math
```

```
from collections import Counter
from nltk.util import ngrams

corpus = [
    "machine learning improves business",
    "machine learning enables automation",
    "machine learning drives innovation"
]

# Tokenization
sentences = [sentence.lower().split() for sentence in corpus]

tokens = [word for sentence in sentences for word in sentence]

# Unigram, Bigram and Trigram counts
unigram_counts = Counter(tokens)

bigram_counts = Counter(
    bigram for sentence in sentences
    for bigram in ngrams(sentence, 2)
)

trigram_counts = Counter(
    trigram for sentence in sentences
    for trigram in ngrams(sentence, 3)
)

# Q1: Bigram probability
p_learning_machine = (
    bigram_counts[("machine", "learning")]
```

```
    / unigram_counts["machine"]
)

print("P(learning | machine) =", p_learning_machine)

# Q3: Deleted interpolation
p_trigram = (
    trigram_counts[("machine", "learning", "improves")]
    / bigram_counts[("machine", "learning")]
)

p_bigram = (
    bigram_counts[("learning", "improves")]
    / unigram_counts["learning"]
)

p_unigram = unigram_counts["improves"] / len(tokens)

lambda1 = 0.5
lambda2 = 0.3
lambda3 = 0.2

p_interpolation = (
    lambda1 * p_trigram +
    lambda2 * p_bigram +
    lambda3 * p_unigram
)

print("Trigram probability =", p_trigram)
print("Bigram probability =", p_bigram)
```

```
print("Unigram probability =", p_unigram)
print("Deleted Interpolation probability =", p_interpolation)

# Q4: Entropy
probabilities = np.array([0.33, 0.33, 0.33])

entropy = -sum(
    p * math.log2(p)
    for p in probabilities
    if p > 0
)

print("Entropy =", entropy)

# Prediction
prediction = {
    "improves": 0.33,
    "enables": 0.33,
    "drives": 0.33
}

max_probability = max(prediction.values())

print("Most probable next-word candidates:")

for word, probability in prediction.items():
    if probability == max_probability:
        print(word, "->", probability)
```

Expected key results

P(learning | machine) = 1.0

Trigram probability = 0.3333

Bigram probability = 0.3333

Unigram probability = 0.0833

Deleted Interpolation probability ≈ 0.2833

Entropy ≈ 1.58 bits

Since all three prediction probabilities are equal, there is no unique best prediction. The system considers "improves", "enables", and "drives" equally likely.

## CASE STUDY 2 – AI-POWERED HOSPITAL APPOINTMENT CHATBOT

Introduction

A healthcare chatbot uses POS tagging to understand the grammatical role of words before identifying user intent.

The two sentences are:

1. "Book an appointment with the doctor."
2. "The book contains medical information."

The word book is ambiguous because it can function as either a verb or a noun depending on context.

The system uses the Penn Treebank POS tagset and HMM-based probabilistic tagging.

Given:

[
P(book|VB)=0.7
]

[
P(book|NN)=0.3
]

[
P(Start\rightarrow VB)=0.6
]

[
P(Start\rightarrow NN)=0.4
]

### Q1. POS Tagging of the Two Sentences

Sentence 1

"Book an appointment with the doctor."

| Word | POS Tag | Explanation |
|---|---|---|
| Book | VB | Verb/action |
| an | DT | Determiner |
| appointment | NN | Noun |
| with | IN | Preposition |
| the | DT | Determiner |
| doctor | NN | Noun |

Therefore:

Book/VB an/DT appointment/NN with/IN the/DT doctor/NN

Why is "book" a verb?

"Book" represents an action performed by the user.

The sentence means:

Schedule an appointment.

Therefore, book is a VB (Verb, base form).

Sentence 2

"The book contains medical information."

| Word | POS Tag | Explanation |
|---|---|---|
| The | DT | Determiner |
| book | NN | Noun |
| contains | VBZ | Verb, third-person singular |
| medical | JJ | Adjective |
| information | NN | Noun |

Therefore:

The/DT book/NN contains/VBZ medical/JJ information/NN

Why is "book" a noun?

Here, "book" refers to a physical or informational object.

Therefore:

[
\boxed{book=NN}
]

The grammatical context determines the correct POS tag.

Q2. HMM Probability Calculation

The HMM probability is calculated as:

[
P(tag,word)

P(tag|Start)\times P(word|tag)
]

Probability of book as VB

Given:

[
P(Start\rightarrow VB)=0.6
]

[
P(book|VB)=0.7
]

Therefore:

[
P(VB,book)=0.6\times0.7
]

[
\boxed{P(VB,book)=0.42}
]

Probability of book as NN

Given:

[
P(Start\rightarrow NN)=0.4
]

[
P(book|NN)=0.3
]

Therefore:

[
P(NN,book)=0.4\times0.3
]

$$
\boxed{P(NN,book)=0.12}
$$

Comparison

$$
0.42>0.12
$$

Therefore, using the supplied beginning-of-sentence probabilities:

$$
\boxed{VB\ is\ the\ most\ probable\ tag}
$$

Interpretation

The HMM favors VB because both the transition probability from the start state and the emission probability of "book" given VB are higher than the corresponding values for NN.

However, in the second sentence, contextual information identifies "book" as a noun. This demonstrates why HMM tagging normally uses more contextual probabilities than the simplified calculation provided here.

**Q3. Rule-Based POS Tagging vs HMM Tagging**

| Feature | Rule-Based Tagging | HMM Tagging |
|---|---|---|
| Main principle | Hand-written linguistic rules | Probabilities learned from data |
| Context | Uses explicit rules | Uses statistical context |
| Training data | Not necessarily required | Required |
| Ambiguity | Rules must cover ambiguity | Probabilities can rank alternatives |
| Flexibility | Lower for unseen contexts | Higher when trained on representative data |
| Interpretability | High | Moderate |
| Healthcare chatbot | Easy to explain | Better for probabilistic ambiguity |

Rule-Based Approach

For example:

If "book" is followed by "an appointment",

assign VB.

This is interpretable but may require many manually designed rules.

HMM Approach

The HMM uses emission and transition probabilities to select the most probable tag.

It is more suitable when large amounts of tagged training data are available.

Limitations

Rule-based tagging can fail when a sentence does not match a predefined rule.

HMM tagging can fail when:

- training data is insufficient,
- probabilities are inaccurate,
- unseen words occur,
- the model does not capture long-range context.

**Q4. Importance of the Penn Treebank Tagset**

The Penn Treebank tagset provides standardized grammatical categories such as:

- NN – noun
- VB – verb
- JJ – adjective
- DT – determiner
- IN – preposition
- VBZ – third-person singular verb

A standardized tagset helps downstream NLP systems.

1. Intent Detection

POS information can help distinguish an action from an object.

For example:

Book/VB an appointment

strongly indicates an appointment-booking intent.

2. Entity Identification

Nouns can provide useful information for identifying entities.

For example:

doctor/NN

appointment/NN

can help identify important concepts in a healthcare query.

3. Response Generation

The grammatical structure helps the chatbot understand what the user is requesting and generate an appropriate response.

Conclusion

Standardized POS tagging provides a consistent syntactic representation that can support intent detection, entity identification, and response generation.

**Q5. Python Program**

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download("punkt")
nltk.download("averaged_perceptron_tagger_eng")
sentences = [
    "Book an appointment with the doctor.",
    "The book contains medical information."
]
print("TOKENIZATION AND POS TAGGING")
print("=" * 50)
for sentence in sentences:
    tokens = word_tokenize(sentence)
    tags = nltk.pos_tag(tokens)
    print("\nSentence:", sentence)
    print("Tokens:", tokens)
    print("POS Tags:", tags)
p_book_vb = 0.7
p_book_nn = 0.3
p_start_vb = 0.6
p_start_nn = 0.4
prob_vb = p_start_vb * p_book_vb
prob_nn = p_start_nn * p_book_nn
print("\nHMM PROBABILITY CALCULATION")
print("=" * 50)
```

```
print("P(Start -> VB) =", p_start_vb)
print("P(book | VB) =", p_book_vb)
print("P(VB, book) =", prob_vb)
print("\nP(Start -> NN) =", p_start_nn)
print("P(book | NN) =", p_book_nn)
print("P(NN, book) =", prob_nn)
if prob_vb > prob_nn:
    print("\nMost probable tag for book at sentence start: VB")
else:
    print("\nMost probable tag for book at sentence start: NN")
```

Expected HMM result

P(VB, book) = 0.42

P(NN, book) = 0.12

Most probable tag = VB

**CASE STUDY 3 – FINANCIAL NEWS POS TAG CORRECTION AND UNCERTAINTY ANALYSIS**

**Introduction**

A financial news analytics system initially assigns the following tags:

market/NN

growth/NN

drives/NNS

investment/NN

The sentence is:

"Market growth drives investment."

A transformation-based tagging rule is provided:

Change NNS to VBZ if the preceding word is tagged as NN.

The corpus frequencies are:

| Word | Frequency |
|---|---|
| market | 500 |

| Word | Frequency |
| --- | --- |
| growth | 350 |
| drives | 180 |
| investment | 420 |

**Q1. Apply the Transformation Rule**

Initial tagging:

market/NN

growth/NN

drives/NNS

investment/NN

The rule is:

Change NNS to VBZ if the preceding word is tagged as NN.

The word before drives is:

growth/NN

Therefore, the rule applies to drives.

Corrected sequence

market/NN

growth/NN

drives/VBZ

investment/NN

Therefore:

$$
\boxed{drives/VBZ}
$$

is the corrected tag.

Linguistic justification

"Market growth" forms the subject of the sentence.

"drives" is the main verb.

"investment" is the object.

The sentence structure is approximately:

Subject Verb Object

Market growth drives investment

Therefore, drives should receive the VBZ tag.

**Q2. Analysis of the Error in "drives"**

The original system assigned:

drives/NNS

NNS means plural noun.

However, in the sentence:

Market growth drives investment.

drives expresses an action performed by the subject "market growth".

Therefore, it is a verb.

Because the subject is singular and the sentence is in the present tense, the appropriate Penn Treebank tag is:

[
\boxed{VBZ}
]

Contextual evidence

Market/NN

growth/NN

drives/VBZ

investment/NN

The surrounding words help identify the grammatical role.

The noun phrase:

market growth

acts as the subject, and drives functions as its verb.

**Q3. Relative Frequency / Probability**

The total frequency is:

[
500+350+180+420=1450
]

Therefore:

Market

$$
P(market)=\frac{500}{1450}
$$

$$
\boxed{P(market)=0.3448}
$$

Growth

$$
P(growth)=\frac{350}{1450}
$$

$$
\boxed{P(growth)=0.2414}
$$

Drives

$$
P(drives)=\frac{180}{1450}
$$

$$
\boxed{P(drives)=0.1241}
$$

Investment

$$
P(investment)=\frac{420}{1450}
$$

$$
\boxed{P(investment)=0.2897}
$$

Probability table

| Word | Frequency | Relative Frequency |
|---|---|---|
| market | 500 | 0.3448 |
| growth | 350 | 0.2414 |
| drives | 180 | 0.1241 |
| investment | 420 | 0.2897 |
| Total | 1450 | 1.0000 |

Interpretation

The word "market" has the highest relative frequency, followed by "investment", "growth", and "drives".

Corpus frequencies can help probabilistic NLP systems estimate how likely words are and can provide statistical evidence during language modeling and tagging.

**Q4. Entropy of the Word-Frequency Distribution**

Entropy is:

$$
H=-\sum p(x)\log_2p(x)
$$

The probabilities are:

$$
0.3448,\ 0.2414,\ 0.1241,\ 0.2897
$$

Therefore:

$$
\begin{aligned}
H=&\\
&-[0.3448\log_2(0.3448)\\
&+0.2414\log_2(0.2414)\\
&+0.1241\log_2(0.1241)\\
&+0.2897\log_2(0.2897)]
\end{aligned}
$$

The result is approximately:

$$
\boxed{H=1.9161\ \text{bits}}
$$

Before and after transformation

The transformation changes the POS tag of drives:

NNS → VBZ

However, it does not change the word frequencies.

Therefore, the word-frequency distribution remains:

market = 500

growth = 350

drives = 180

investment = 420

Hence:

$$
\boxed{H_{before}=1.9161\ bits}
$$

and

$$
\boxed{H_{after}=1.9161\ bits}
$$

Interpretation

The entropy measures uncertainty in the supplied word-frequency distribution.

Since the transformation only changes the POS tag and not the word frequency, it does not change the entropy of the word-frequency distribution.

The POS tagging uncertainty itself may improve because the incorrect NNS tag has been corrected to VBZ.

**Q5. Python Program**

```
import nltk

import math

from collections import Counter

nltk.download("punkt")

nltk.download("averaged_perceptron_tagger_eng")

sentence = "Market growth drives investment."

tokens = nltk.word_tokenize(sentence)

print("TOKENS")

print(tokens)

initial_tags = [

    ("Market", "NN"),

    ("growth", "NN"),

    ("drives", "NNS"),

    ("investment", "NN")

]

print("\nINITIAL POS TAGS")
```

```
for word, tag in initial_tags:
    print(word + "/" + tag)
corrected_tags = initial_tags.copy()
for i in range(1, len(corrected_tags)):
    current_word, current_tag = corrected_tags[i]
    previous_word, previous_tag = corrected_tags[i - 1]

    if current_tag == "NNS" and previous_tag == "NN":
        corrected_tags[i] = (current_word, "VBZ")

print("\nCORRECTED POS TAGS")
for word, tag in corrected_tags:
    print(word + "/" + tag)

# Frequency analysis
frequencies = {
    "market": 500,
    "growth": 350,
    "drives": 180,
    "investment": 420
}
total = sum(frequencies.values())
print("\nWORD FREQUENCIES AND PROBABILITIES")
probabilities = {}
for word, frequency in frequencies.items():
    probability = frequency / total
    probabilities[word] = probability
    print(
        word,
        "Frequency =", frequency,
```

```
        "Probability =", round(probability, 4)
    )
entropy = 0
for probability in probabilities.values():
    entropy -= probability * math.log2(probability)
print("\nTotal frequency =", total)
print("Entropy =", round(entropy, 4), "bits")
```

Expected corrected POS output

Market/NN

growth/NN

drives/VBZ

investment/NN

Expected probability output

market = 0.3448

growth = 0.2414

drives = 0.1241

investment = 0.2897

Expected entropy

Entropy = 1.9161 bits

## CONCLUSION

The three case studies demonstrate important NLP techniques used for practical language-processing systems.

Case Study 1

N-gram models can support e-commerce search prediction. MLE estimates probabilities from corpus frequencies, while Backoff allows the system to fall back to lower-order models for unseen sequences. Deleted Interpolation combines trigram, bigram, and unigram information. Entropy measures uncertainty in the prediction distribution.

Case Study 2

POS tagging is important for understanding user queries in a healthcare chatbot. The word "book" demonstrates contextual ambiguity because it can be a verb or a noun. Rule-based and HMM-based approaches provide different methods for resolving this ambiguity.

Case Study 3

Transformation-Based Tagging can correct POS-tagging errors by applying linguistic rules. In the sentence "Market growth drives investment", the incorrect NNS tag for "drives" is corrected to VBZ. Corpus frequency and entropy provide additional statistical analysis of the language data.

Overall, these case studies demonstrate how N-gram models, smoothing/backoff, entropy, POS tagging, HMMs, and transformation-based tagging can be applied to real-world NLP systems.